

Using Amazon S3 to Handle Large Items in DynamoDB To Begin with the Lab

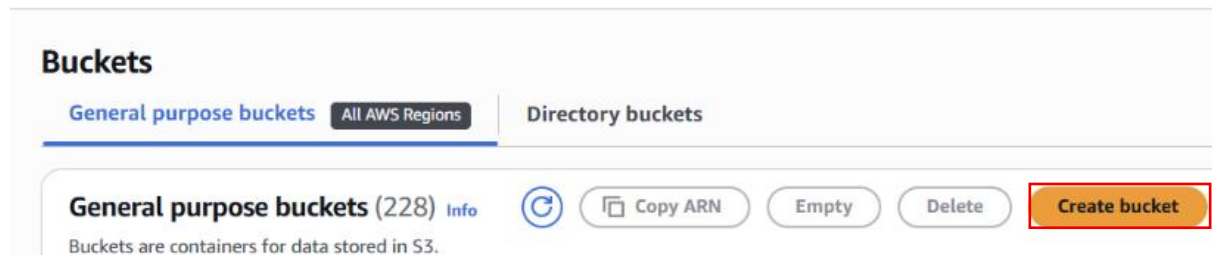
Summary of the Lab

This lecture explains how to handle large DynamoDB item attributes by storing them in Amazon S3 instead of using Gzip compression. It covers creating an S3 bucket, assigning IAM permissions, uploading content to S3, saving the S3 URL in DynamoDB, and retrieving the content back from S3. This approach is ideal for storing large files such as images, audio, and videos.

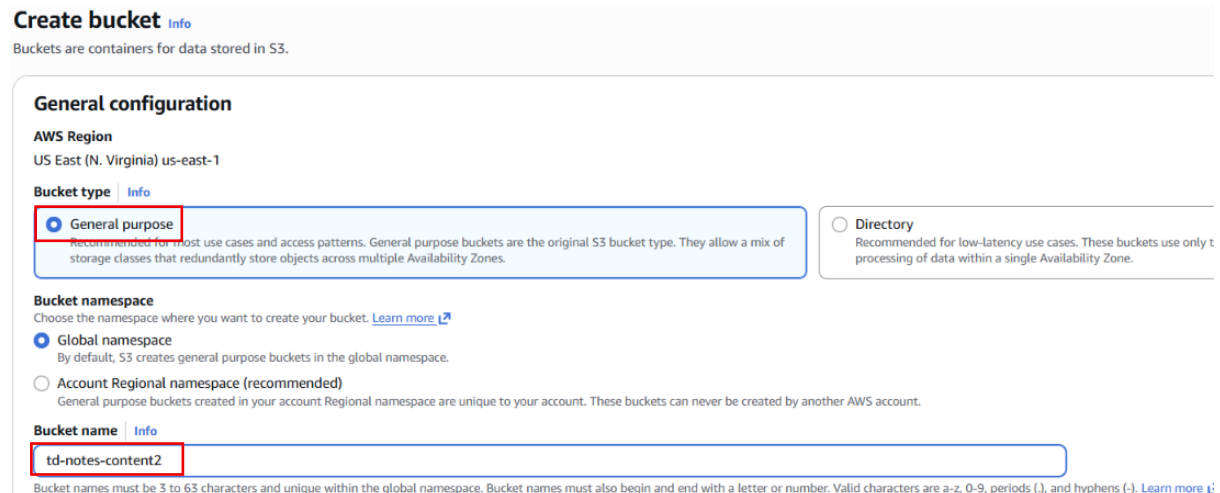
Lab steps

Step 1: Create an S3 Bucket

- Go to Amazon S3 in AWS Account
- Following the steps to Create bucket



- Select General Purpose Bucket type
- Provide the name to identify the bucket



- Choose the Block all public access
- Disable the Bucket versioning

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its contents is controlled, you can turn on Block all public access. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will not require public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☒ Block all public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☒ Block public access to buckets and objects granted through new access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing public access permissions.

☒ Block public access to buckets and objects granted through any access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

☒ Block public access to buckets and objects granted through new public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☒ Block public and cross-account access to buckets and objects through any public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your bucket, even if you delete an object accidentally. You can use versioning to preserve, retrieve, and restore every version of every object stored in your bucket, even if you delete an object accidentally. [Learn more](#)

Bucket Versioning

☒ Disable

☐ Enable

- Go to Default encryption section
- Choose Encryption type, **Server-side encryption with AmazonS3 managed keys (SSE-S3)**
- Enable the **Bucket Key**
- Click on the **Create Bucket**

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type

Secure your objects with two separate layers of encryption. For details on pricing, see DSSE-KMS pricing on the Storage tab of the [Amazon S3 pricing page](#).

☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)

☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)

☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

☐ Disable

☒ Enable

- Bucket is Created

General purpose buckets (229)

Buckets are containers for data stored in S3.

td-notes	1 match	< 1 >	⚙
Name	AWS Region	Creation date	
☒ td-notes-content2	US East (N. Virginia) us-east-1	July 7, 2026, 10:34:21 (UTC+05:30)	

Step 2: Create a DynamoDB Table

- Go to DynamoDB in AWS Account
 - Under DynamoDB go to Tables
 - Create a table
- ### Tables (4)

Find tables

Filter by tag: Any tag key

Filter by tag value: Any tag value

Last updated: July 6, 2026, 10:52 (UTC+5:30)

Actions Delete Create table

< 1 >
- Provide the Table Name **global_td-notes** to identify the Table
 - Provide the **partition key** for scalability and availability
 - Provide a **sort key** for allows you to sort or search among all items
 - Click on **Create Table**

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

global_td-noted

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

user_id

1 to 255 characters and case sensitive.

String

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

timestamp

1 to 255 characters and case sensitive.

String

- Table is Created

Tables (5) Info

Find tables

Filter by tag
Any tag key

Filter by tag value
Any tag value

Last updated
July 11, 2026, 09:54 (UTC+5:30)

Actions

Delete

Create table

< 1 >

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode
☐	Employees	Active	EmpId (N)	-	0	0	Off	☆	Provisioned (5)	Provisioned (5)
☐	global_td-notes	Active	user_id (S)	timestamp (S)	0	0	Off	☆	On-demand	On-demand

Step 3: Create IAM Role

- Go to the IAM in AWS Account
- Then go to **IAM User**, select own **user**
- Click on the **Add Permission**

[Permissions](#) | [Groups](#) | [Tags](#) | [Security credentials](#) | [Last Accessed](#)

Permissions policies (3)

Permissions are defined by policies attached to the user directly or through groups. [Learn more](#)

Filter by Type
All types

☐ Policy name

Type

Attached via

[Remove](#) [Add permissions](#) [Add permissions](#) [Create inline policy](#)

- Choose the **Attach policy directly** in Add permission
- Go to the search bar select two policies
 - 1.AmazonS3FullAccess
 - 2.AmazonDynamoDBFullAccess

Add permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, inline policies, and any existing permissions boundaries from an existing user.

☒ **Attach policies directly**

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (2/2670)

Filter by Type

All types

< 1 2 3 4 5 6 7 ... 134 >

- These are the two policies that you are select
- Click on the **Add permission**

Review

The following policies will be attached to this user. [Learn more](#)

User details

User name
aws-aman

Permissions summary (2)

Name	Type	Used as
AmazonS3FullAccess	AWS managed	Permissions policy
AmazonDynamoDBFullAccess	AWS managed	Permissions policy

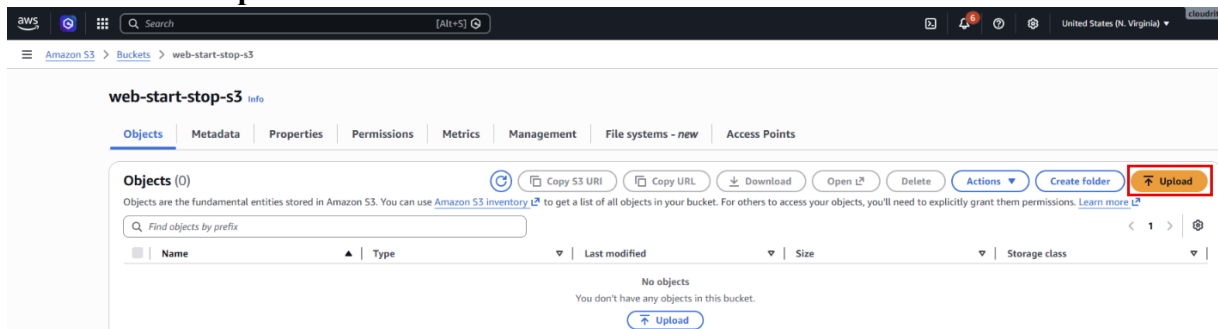
Cancel

Previous

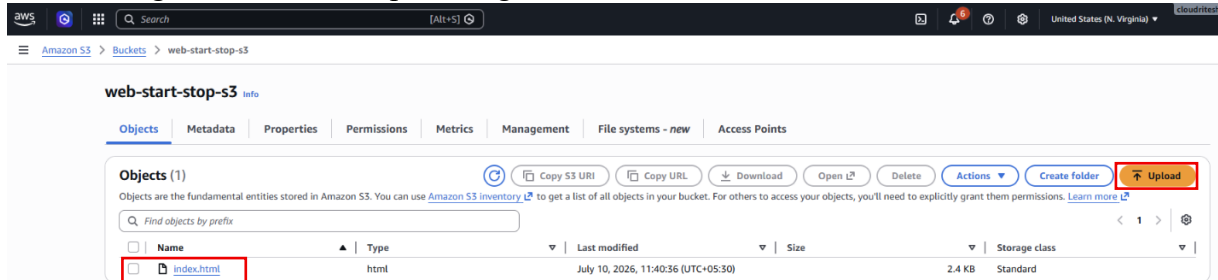
Add permissions

Step 4: Upload a File to S3

- Click on the **upload**

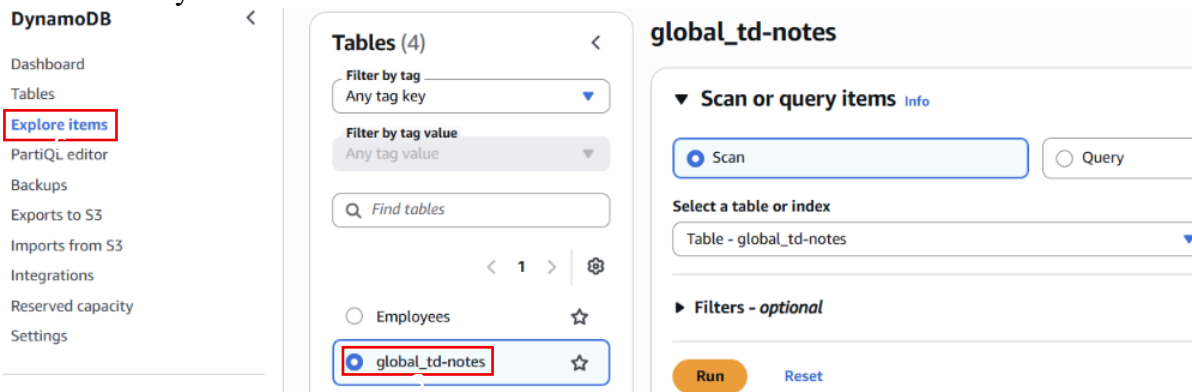


- Click on the **Add file**
- Add large file like media, pdf, image, video, or text file



Step 5: Create an Item in DynamoDB

- Go to the DynamoDB in AWS Account
- Under DynamoDB, click on the **Explore items**
- Select the your Table



- Click on the **create item**
- Paste this **json code**
- Copy the S3_key

```
{
  "user_id": {"S": "89314e66-3721-472d-942c-14387eab5f5e"},
  "timestamp": {"S": "1783420515"},
  "cat": {"S": "Industrial"},
  "title": {"S": "Devolved clear-thinking intranet"},
  "content": {"S": "Try to copy the FTP pixel, maybe it will calculate the solid state microchip!"},
  "note_id": {"S": "d31acdf7-331c-467c-baf7-3fe33fa95996"},
  "user_name": {"S": "Gracie_Kub61"},
  "S3_key": {"S": "notes.txt"},
  "expires": {"N": "1783421115"}
}
```

Create item Form JSON view

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attributes View DynamoDB JSON Copy

```
1 {
2   "user_id": {"S": "89314e66-3721-472d-942c-14387eab5f5e"},
3   "timestamp": {"S": "1783420515"},
4   "cat": {"S": "Industrial"},
5   "title": {"S": "Devolved clear-thinking intranet"},
6   "content": {"S": "Try to copy the FTP pixel, maybe it will calculate the solid state microchip!"},
7   "note_id": {"S": "d31acdf7-331c-467c-baf7-3fe33fa95996"},
8   "user_name": {"S": "Gracie_Kub61"},
9   "S3_key": {"S": "notes.txt"},
10  "expires": {"N": "1783421115"}
11 }
```

- Item is created

Table: global_td-notes - Items returned (1) Actions Create item

Scan started on July 11, 2026, 10:34:41

	user_id (String)	timestamp (String)	cat	content	expires	key_S3	note_id	title	user_name
☐	89314e66-3721-472...	1783420515	Industrial	Try to copy ...	1783421115	notes.txt	d31acdf7-3...	Devolved cl...	Gracie_Kub61

Step 6: Retrieve Metadata from DynamoDB

- Search the code on the AI Application
- Implement code for S3 + DynamoDB Large Item Storage using AWS SDK for JavaScript v3
- Create a File **large-item-S3.js**, open with Vs code
- Paste this code in large-item-S3 file
- Download the file in Resources

```

15 large-items-S3.js > ...
1  const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");
2  const {
3    DynamoDBDocumentClient,
4    PutCommand,
5    GetCommand,
6  } = require("@aws-sdk/lib-dynamodb");
7  const {
8    S3Client,
9    PutObjectCommand,
10   GetObjectCommand,
11  } = require("@aws-sdk/client-s3");
12  const faker = require("faker");
13  const moment = require("moment");
14
15  const REGION = "us-east-1";
16  const TABLE_NAME = "global_td-notes";
17  const BUCKET_NAME = "td-notes-content2";
18
19  // DynamoDB Client
20  const dynamoClient = new DynamoDBClient({
21    region: REGION,
22  });
23
24  const docClient = DynamoDBDocumentClient.from(dynamoClient);
25
26  // S3 Client
27  const s3Client = new S3Client({
28    region: REGION,
29  });
30
31  (async () => {
32    try {
33      const item = generateNotesItemS3();
34
35      console.log("Original Item");
36      console.log(item);
37
38      await putNotesItemS3(item);
39
40      console.log("\nStored Item");
41
42      const data = await getNotesItemS3({
43        user_id: item.user_id,
44        timestamp: item.timestamp,
45      });
46
47      console.log("\nRetrieved Item");
48      console.log(data.Item);
49    } catch (err) {
50      console.error("ERROR:");
51    }

```

- Go to the Terminal
- If AWS SDK is not Install, installed the AWS SDK
- Implement this command Run in the terminal, **node Large-Item-S3.js**
- Item is stored successfully
- Amazon S3 is use to store large scale item like Media object

```
PS C:\Users\rrath\OneDrive\Documents\Sandbox> node large-items-S3.js
Original Item:
{
  user_id: '89314e66-3721-472d-942c-14387eab5f5e',
  timestamp: '1783420515',
  cat: 'Industrial',
  title: 'Devolved clear-thinking intranet',
  content: 'Try to copy the FTP pixel, maybe it will calculate the solid state microchip!',
  note_id: 'd31acdf7-331c-467c-baf7-3fe33fa95996',
  user_name: 'Gracie_Kub61',
  expires: 1783421115
}

Item stored successfully.

Retrieved Item:
{
  user_id: '89314e66-3721-472d-942c-14387eab5f5e',
  expires: 1783421115,
  cat: 'Industrial',
  note_id: 'd31acdf7-331c-467c-baf7-3fe33fa95996',
  user_name: 'Gracie_Kub61',
  title: 'Devolved clear-thinking intranet',
  timestamp: '1783420515',
  content: 'Try to copy the FTP pixel, maybe it will calculate the solid state microchip!'
}
PS C:\Users\rrath\OneDrive\Documents\Sandbox> 
```